

Walchand College Of Engineering, Sangli.

(An Autonomous Institute)

Department Of
Computer Science and Engineering

A Project Report
On

EVO-X: An Autonomous AI Model Evolution Framework for Continual Intrusion Detection under Concept Drift

Submitted by

Mr. Prshil Vilas Suryawanshi	**(23510006)**
Ms. Rameshwari Rajendra Satpute	**(23510007)**
Ms. Pooja Amol Bhosale	**(23510012)**
Mr. Himanshu Nitin Bangalkar	**(23510033)**

Under the Guidance
of

Prof. Leena V. Patil

Year: 2026 - 2027



Walchand College of Engineering, Sangli

(An Autonomous Institute)

Department
Of
Computer Science and Engineering

CERTIFICATE

This is to certify that the Project Report entitled, **"EVO-X: An Autonomous AI Model Evolution Framework for Continual Intrusion Detection under Concept Drift"** submitted by

Mr. Prshil Vilas Suryawanshi	**(23510006)**
Ms. Rameshwari Rajendra Satpute	**(23510007)**
Ms. Pooja Amol Bhosale	**(23510012)**
Mr. Himanshu Nitin Bangalkar	**(23510033)**

to Walchand College of Engineering, Sangli, is a record of bona fide Project work carried out by them under my supervision and guidance and is worthy of consideration for the award of the degree of Bachelor of Technology in Computer Science and Engineering of the Institute.

Prof. Leena V. Patil
Guide

External Examiner

Dr. A. R. Surve
Head
Department of Computer Science and Engineering

Acknowledgement

We feel immense pleasure in submitting this project report entitled **"EVO-X: An Autonomous AI Model Evolution Framework for Continual Intrusion Detection under Concept Drift"**.

We are thankful to our guide, **Prof. Leena V. Patil**, for the valuable guidance and kind help during the completion of the project.

We are also thankful to the Head of the Department of Computer Science and Engineering, **Dr. A. R. Surve**, for the valuable guidance during the completion of the project.

We are grateful to Mr. Anish Shobith P S, Master's student (Cyber Security) at the Manipal School of Information Sciences, for reviewing our project synopsis and giving us expert feedback that sharpened the research direction of the controller, the experiments and the baselines. **[TODO: confirm that he agrees to be named]**

We would like to thank all the faculty members and staff of the Department of Computer Science and Engineering for their generous help in various ways in the completion of this project.

We would like to thank all our friends, and especially our classmates, for the thoughtful and mind-stimulating discussions we had, which prompted us to think beyond the obvious. We have enjoyed their companionship so much during our stay at Walchand College of Engineering, Sangli.

Declaration

We hereby declare that work presented in this project report titled **"EVO-X: An Autonomous AI Model Evolution Framework for Continual Intrusion Detection under Concept Drift"** submitted by us in the partial fulfillment of the requirement of the award of the degree of **Bachelor of Technology (B.Tech)** submitted in the **Department of Computer Science and Engineering, Walchand College of Engineering, Sangli**, is an authentic record of our project work carried out under the guidance of Prof. Leena V. Patil, Assistant Professor, Department of Computer Science and Engineering, Walchand College of Engineering, Sangli.

Mr. Prshil Vilas Suryawanshi (23510006)

Ms. Rameshwari Rajendra Satpute (23510007)

Ms. Pooja Amol Bhosale (23510012)

Mr. Himanshu Nitin Bangalkar (23510033)

Date : (Signatures)

Place : Sangli

ABSTRACT

Machine-learning intrusion detection systems (IDS) lose accuracy after deployment because attacker techniques and legitimate traffic change over time, a phenomenon called concept drift. Existing continual-learning IDS provide adaptation mechanisms, but they usually apply one fixed update pipeline whenever drift is detected and rarely verify an updated model before it replaces the deployed one. This project presents EVO-X, a model-evolution management framework built around a replaceable flow-based classifier (Random Forest or XGBoost). EVO-X detects drift with ADWIN, Page–Hinkley, PSI and MMD, characterises it by magnitude, persistence, unfamiliarity and recurrence, and uses a two-stage controller, eligibility rules followed by multi-objective utility ranking, to choose among five adaptation strategies. The resulting candidate model is evaluated on independent recent and historical holdouts and must pass a fail-closed safety gate before promotion. Promoted models are versioned in a tamper-evident registry, monitored, and restored to the parent version if they degrade. The framework is evaluated by streaming CICIDS2017 flows with controlled injected drift (six scenario types plus a stationary control) against nine alternative policies. A first quick run on real CICIDS2017 flows (seven scenarios, one test seed) showed that the fixed-update baselines deployed two updates per scenario with outcomes ranging from improved to clearly harmful, while EVO-X made no promotions and matched the static model. All four research hypotheses are therefore inconclusive, and full multi-seed experiments are required before any performance claim is made.

Table Of Contents

1 Introduction **11**

1.1 Problem Statement . . . 12

1.2 Motivation . . . 13

1.3 Significance of the problem . . . 13

2 Literature Review , Gap Identified **15**

2.1 Background . . . 15

2.2 Related Work . . . 16

2.3 Gap Identified . . . 17

3 Customer Details and Use Cases **18**

3.1 Customer Details . . . 18

3.2 Feedback Received and How It Was Addressed . . . 18

3.3 Use Cases . . . 19

4 Project Objectives **20**

5 Proposed Method/Algorithm **21**

5.1 Problem Definition . . . 21

5.2 Proposed Idea/System . . . 22

5.3 System Architecture . . . 22

5.4 System Analysis and Design . . . 26

5.4.1 Requirement Specification . . . 26

5.4.2 Flowcharts / DFDs / UML / Functional Block Diagram . . . 26

5.4.3 Design and Test Cases . . . 27

5.4.4 Algorithms and Pseudo Code . . . 27

5.4.5 UI/UX Wireframes where applicable . . . 28

6 Performance Study **29**

6.1 Implementation/Simulation Environment . . . 29

6.2 Results and Analysis . . . 32

6.3 Summary of performance study . . . 35

6.4 Limitations . . . 35

7 Conclusions and Future Work **37**

8 Scholarly Contributions **38**

9 References **39**

A Appendix A: Plagiarism Report, AI Similarity Report **41**

List of Figures

1 Shapes of concept drift: the vertical axis is the degree of change in traffic behaviour relative to the training data and the horizontal axis is time. 11

2 Conventional "drift detected, so retrain" reflex versus the governed evolution loop of EVO-X. 12

3 End-to-end architecture of EVO-X. 23

4 Role assignment of source flows. Each original flow belongs to exactly one role. 25

5 Use-case diagram of EVO-X. 28

6 Class diagram of the main components. 29

7 Activity diagram of one evolution cycle. 42

8 Sequence diagram of drift detection through promotion or rollback. 43

9 Wireframe of the planned dashboard. 43

10 Quick-run comparison across the ten policies: reporting macro-F1 (top left), historical forgetting (top right), promoted updates (bottom left) and training plus proxy-search time in seconds (bottom right). Bars show the mean over seven scenarios; error bars show spread across scenarios. 44

List of Tables

1 Summary of what existing work demonstrates and what is left open. 17

2 Customer / expert-consultation details. 18

3 Expert feedback and corresponding action. 18

4 Illustrative proxy estimates for the worked example (invented for explanation only). 25

5 Functional requirements. 26

6 Non-functional requirements. 27

7 Representative test cases. 30

8 Parameters varied in the experiments. 32

9 Safety-gate thresholds selected on validation data and frozen for the quick run. 32

10 Quick-run results on real CICIDS2017 flows with injected drift (mean over seven scenarios, one test seed, Random Forest). Preliminary. 33

Acronyms

- IDS Intrusion Detection System
- ML Machine Learning
- ADWIN Adaptive Windowing (drift detector)
- PH Page–Hinkley test
- PSI Population Stability Index
- MMD Maximum Mean Discrepancy
- FPR / FNR False Positive / False Negative Rate
- F1 Harmonic mean of precision and recall
- SSF Strategic Selection and Forgetting
- RF Random Forest
- CICIDS2017 Canadian Institute for Cybersecurity IDS 2017 dataset
- SOC Security Operations Centre
- MLOps Machine-learning operations (model lifecycle management)

1 Introduction

This chapter introduces the problem that the project addresses, explains why it matters, and summarises the approach, the experiments and what has been found so far. Network intrusion detection systems (IDS) watch network traffic for malicious activity. Over the last decade, machine learning has become the dominant way to build them: instead of matching fixed attack signatures, a classifier is trained on labelled traffic records such as CICIDS2017 [8] or UNSW-NB15 [9] and then used to label new traffic as benign or as a specific attack type. The usual pipeline is to collect historical data, train a model, deploy it and predict attacks. That pipeline silently assumes that the relationship between network features and attack labels stays the same after deployment.

In practice the assumption fails. Attackers keep changing their tools, timing and payloads to evade detection, and legitimate behaviour changes as organisations adopt new applications and devices. When the joint distribution of inputs and labels changes after a model has been trained, the situation is called *concept drift* [1]. The literature distinguishes abrupt, gradual, incremental and recurring drift, and Figure 1 sketches these shapes together with the temporary shift that this project also studies. A model that is never updated slowly loses detection capability, and nobody notices unless the deployment actively looks for it.

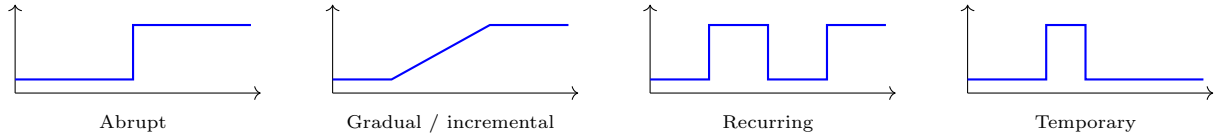


Figure 1: Shapes of concept drift: the vertical axis is the degree of change in traffic behaviour relative to the training data and the horizontal axis is time.

The common response to drift is retraining. Retraining has a cost in computing time and validation effort, and it carries a risk: a model retrained only on recent data can forget older attacks that are still relevant (catastrophic forgetting), and a badly retrained model can be worse than the one it replaces. Recent research on continual-learning IDS, discussed in Chapter 2, shows good mechanisms for adapting a model. However, those mechanisms are usually applied in one fixed way whenever drift is flagged, and the updated model is usually put straight into service.

This project proposes **EVO-X**, a framework that treats the evolution of a deployed IDS as a governed decision process instead of an automatic reflex. Figure 2 contrasts the conventional approach with the EVO-X approach. In EVO-X, drift is detected and characterised, a controller chooses one of five adaptation strategies, the strategy produces a *candidate* model that is never deployed directly, the candidate

is tested on independent recent and historical data, a safety gate approves or rejects it, and an approved model is monitored after deployment and rolled back if it degrades. The classifier itself (a Random Forest or XGBoost model working on network-flow features) is a replaceable component and is not the contribution of the project; the contribution is the governance layer around it.

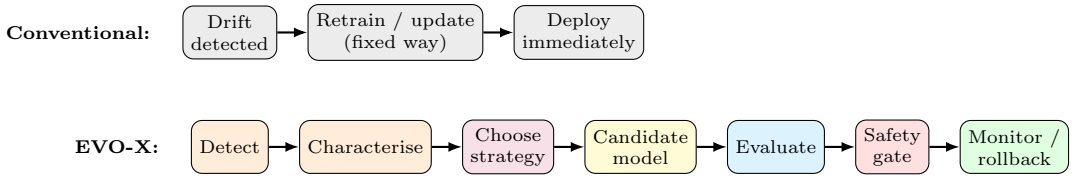


Figure 2: Conventional "drift detected, so retrain" reflex versus the governed evolution loop of EVO-X.

1.1 Problem Statement

Static machine-learning IDS models degrade when network traffic evolves, because concept drift introduces new attack patterns and changes normal behaviour. Periodic retraining is expensive and can cause catastrophic forgetting of previously learned attacks. Existing adaptive IDS apply a fixed adaptation strategy once drift is detected, without considering the type, severity or persistence of the drift, and they often lack a safety mechanism to validate an updated model before deployment. The problem addressed here is to design and evaluate an autonomous IDS that detects, characterises and responds to concept drift by selecting a suitable adaptation strategy and promoting an updated model only if it is demonstrably safe.

The problem is hard for several reasons. Drift must be detected from signals that are partly unavailable at run time, because true labels arrive late or not at all. Different drifts call for different responses: a brief blip should be ignored, a recurring pattern can be handled with memory of the past, and a new attack family may need a fresh model. Updates must balance plasticity (learning the new) against stability (keeping the old), and the quality of an update can be judged only on data that the update did not train on. In terms of the state of the art, mechanisms for drift detection and continual learning are well developed, while the orchestration of several adaptation options with an explicit candidate safety gate and rollback is comparatively less studied for network-flow IDS.

1.2 Motivation

The motivation for this project has three parts. First, *operational realism*: in real security operations a deployed model is not retrained blindly on every new batch of data, because retraining costs compute, downtime and validation effort and a bad retraining can make things worse. Mature machine-learning operations (MLOps) practice already manages other production models with versioning, validation before release and rollback, but this discipline is largely absent from the academic continual-learning IDS literature.

Second, *forgetting is a safety risk*. An IDS that adapts to new attacks but forgets older, still-relevant ones may be more dangerous than one that does not adapt at all, because operators may believe protection has improved when it has not. A framework that explicitly measures and gates against forgetting addresses a practical blind spot. Third, the project has *educational and research value*: it requires concept-drift theory, continual learning, multi-objective decision making, streaming systems and model lifecycle practice together, at a depth appropriate for a final-year research project that a small team can complete in one academic year.

The people who should care about this work are security operations teams who run learning-based detectors, researchers in adaptive and continual-learning IDS, and engineers responsible for keeping production machine-learning models safe and auditable.

1.3 Significance of the problem

The approach of EVO-X is to separate the question *how to adapt* from the question *whether and which adaptation to apply*. The hypothesis behind the design is that choosing the adaptation according to the measured nature of the drift, and refusing to deploy any update that fails an independent check, will give fewer unnecessary and fewer harmful updates than always applying the same update, at comparable detection quality. Existing systems are not adequate for this purpose because, as Chapter 2 explains, they provide strong update mechanisms but a single fixed pipeline and little or no candidate validation. EVO-X is not claimed to invent drift detection or continual learning; the novel element is the combination of multi-strategy selection, independent candidate evaluation, a fail-closed safety gate and monitored rollback for network-flow IDS.

The performance study streams real CICIDS2017 flows with controlled, injected drift of six types (abrupt, gradual, recurring, temporary, incremental and novel-class) and a drift-free control, and compares ten policies: a static model, four fixed update policies, one SSF-inspired policy, an active-learning-inspired

policy, EVO-X and three EVO-X ablations. The measured quantities include macro-averaged F1, historical forgetting, false-positive change, number of updates, training and proxy-search time, and counts of harmful and unnecessary updates. The first quick run on real flows (seven scenarios, one test seed) shows that updating is not automatically beneficial: some fixed-update policies improved macro-F1 over the static model by up to about four points while others lowered it and caused much more forgetting, and EVO-X, which made no promotions in this run, matched the static model without forgetting but showed no gain. The hypotheses are therefore inconclusive at this stage, and the report states this plainly. The full multi-seed experiment is the remaining step needed to decide them.

2 Literature Review , Gap Identified

This chapter first gives the background needed to understand the work and then reviews related systems, ending with the gap that EVO-X targets.

2.1 Background

Flow-based IDS. A network flow summarises a conversation between two hosts, for example by its duration, packet and byte counts, inter-arrival times and flag counts. Flow-based IDS classify each flow, which keeps the input small and independent of packet payloads. CICIDS2017 [8] provides labelled flows for benign traffic and attacks such as DoS, DDoS, brute force, infiltration, botnet and web attacks; UNSW-NB15 [9] and CSE-CIC-IDS2018 [10] offer different feature sets and attack families. These benchmarks are widely used but are known to contain labelling and generation artefacts, so results on them are best read as comparisons between methods rather than as absolute real-world performance.

Concept drift. Gama et al. [1] survey concept-drift adaptation, define the abrupt, gradual, incremental and recurring types, and separate *detect-and-adapt* strategies from continuous adaptation. Drift can affect the input distribution (covariate shift), the class priors (prior shift) or the relation between features and labels. Detection can use the model's error stream, which needs labels, or can compare feature distributions, which does not.

Drift detectors used in this project. ADWIN [11] keeps a variable-length window over a stream and shrinks it when two sub-windows have significantly different means; it carries a theoretical bound on false positives. The Page–Hinkley test [12] accumulates deviations from the running mean and signals a sustained increase; it is light and suits abrupt change. The Population Stability Index (PSI) compares the distribution of each feature in a recent window against reference bins; by convention, values above 0.1 indicate moderate and above 0.25 major shift. Maximum Mean Discrepancy (MMD) [13] is a kernel two-sample statistic that can reveal multivariate changes, such as a change in correlation, that per-feature comparisons miss.

Continual learning and forgetting. Wang et al. [2] describe continual learning as a stability–plasticity trade-off: a model must stay plastic enough to learn new concepts yet stable enough not to forget old ones. Catastrophic forgetting means a sharp loss of performance on earlier classes after training on new data. Common mitigation families are replay (mixing remembered samples into training), regularisation and architectural methods.

2.2 Related Work

Surveys. A 2024 survey on concept drift and feature dynamics in intrusion detection [3] confirms that cybersecurity environments change in ways static IDS struggle with and catalogues detection and adaptation techniques; it informed the design of the drift-detection module.

SSF. Zhang et al. [4] propose a continual-learning method for network IDS that combines drift detection, a strategic sample-selection step that prioritises samples showing the drifted pattern, a strategic forgetting step that discards outdated buffered samples when significant drift is seen, and periodic fine-tuning. It reports state-of-the-art adaptability on NSL-KDD and UNSW-NB15. Its strength is a well-designed adaptation mechanism; its limitation for our purposes is that the same pipeline is applied whenever drift is detected.

RepShield. Liu et al. [5] address forgetting at the optimisation level: a historical feature subspace is built using PCA-based drift detection and new gradient updates are projected onto its orthogonal complement, so new learning interferes less with old knowledge. It also uses strategic sample selection. Again the mechanism is fixed rather than one option among several.

METANOIA. Ying et al. [6] present a lifelong provenance-based intrusion detection and investigation system for host-level attacks such as advanced persistent threats. It treats lifelong operation as a system-design concern, which is closest in spirit to this project, but it targets host and provenance data instead of network flows and has no explicit multi-strategy controller or candidate promotion gate.

Deciding when to retrain. Pham et al. [7] study when a deployed ML system should be retrained and propose a semi-supervised drift-detection technique that reduces labelling effort. This frames retraining as a deliberate decision, in the same spirit as EVO-X, but it is not specific to IDS and does not choose among adaptation types or validate candidates.

Online and active learning. The project proposal also cites a drift-aware hybrid online IDS that combines drift detection with online and active learning to reduce unnecessary labelling and retraining. The exact bibliographic reference could not be verified, so the report describes the corresponding baseline by its mechanism (incremental learning on the least-confident labelled samples) and does not cite it by name. **[TODO: replace with the verified citation or keep as is]**

Table 1: Summary of what existing work demonstrates and what is left open.

System	Demonstrated	Left open (relative to this project)
SSF [4]	Strategic selection and forgetting in one pipeline	One fixed strategy; no candidate gate
RepShield [5]	Gradient projection against forgetting	One fixed mechanism; no strategy choice
METANOIA [6]	Lifelong host-level APT detection	Not network-flow; no controller or gate
Pham et al. [7]	Deciding when to retrain	Not IDS-specific; no strategy choice or safety check

2.3 Gap Identified

It would be wrong to say that no adaptive IDS exists; SSF, RepShield and METANOIA show working adaptive systems, and drift detection and continual learning are established fields. What appears comparatively under-explored for network-flow IDS is a management layer that (i) chooses among qualitatively different adaptation strategies according to the measured characteristics of the drift, (ii) treats the trade-off between performance gain, forgetting, cost and false-positive risk as an explicit multi-objective decision, (iii) evaluates every adaptation as a candidate on both recent and historical data before deployment, (iv) keeps a versioned lifecycle with automatic rollback, and (v) is evaluated for portability across more than one classifier and more than one dataset. This gap has to be re-verified against the newest literature at the time of final submission, because the area moves quickly. **[TODO: record the date and search terms of the final literature check]**

EVO-X is better suited to this gap than the reviewed systems because it keeps their adaptation mechanisms as options (incremental learning, replay, strategic sample selection, full retraining), adds a transparent controller whose every decision is logged, and refuses to deploy a candidate that fails an independent check. Where its contribution overlaps existing work, for instance METANOIA's lifelong-operation framing or general MLOps rollback patterns, the overlap is acknowledged, and the claim is limited to the specific combination applied to network-flow IDS.

3 Customer Details and Use Cases

This chapter records the external expert consulted during the project and describes the situations in which the system is meant to be used.

3.1 Customer Details

Table 2: Customer / expert-consultation details.

Customer name	Anish Shobith P S
Contact person	Anish Shobith P S, Master's student (Cyber Security), Manipal School of Information Sciences
Contact	Website: https://anishshobithps.com/; LinkedIn: linkedin.com/in/anishshobithps/; Email: anish.shobith19@gmail.com
Nature of engagement	Research-oriented academic consultation, contacted through LinkedIn to review the project synopsis and suggest improvements to the research direction and methodology

3.2 Feedback Received and How It Was Addressed

After reading the synopsis, the reviewer said that the orchestration layer, in which drift analysis, multi-strategy adaptation and a pre-deployment safety gate sit in one loop, was the most interesting part, and he pushed on specific points. The visible points of his message and the corresponding project actions are summarised in Table 3. **[TODO: the screenshot shows three of the four points; add the fourth point and its response from the original message]**

Table 3: Expert feedback and corresponding action.

Feedback	Action taken
The selection logic of the Evolution Controller must be spelled out, since it is the core and the least specified part.	The controller was specified as eligibility rules plus utility ranking, with every decision logged (Sections 5.2 and 5.4).
Utility-function weights should come out of experiments instead of being asserted, showing what happens when they move.	Weights and gate thresholds are selected on validation scenarios only from predeclared grids, frozen before testing, and the sensitivity grids are reported.
Benchmark against SSF and HOIDS; a softer comparison will not say much.	An SSF-inspired baseline and an active-learning-inspired baseline are included among ten compared policies.

3.3 Use Cases

The system is intended for the following situations; the use-case diagram is in Section 5.4.2.

1. **SOC engineer keeps a learned detector current.** A security operations team that runs a flow-based ML detector wants it to adapt to new traffic without constant manual retraining, while being confident that no update silently weakens protection. EVO-X proposes updates, validates them and records why each decision was taken.

2. **Researcher compares adaptation policies.** An IDS researcher can plug a new adaptation strategy, drift detector or classifier into the framework and compare it against the fixed policies under the same scenarios, label policy and statistics.

3. **Operator audits model changes.** A reviewer can inspect the evolution history: which drift triggered an update, what the controller chose and why, how the candidate scored, whether the gate promoted it and whether a rollback happened.

4. **Safe experimentation.** A team that wants to try a more aggressive update can run it through the gate and the post-deployment monitor, so that a harmful change is rejected or reverted instead of reaching production.

4 Project Objectives

1. To build a streaming evaluation environment that replays benchmark intrusion data in time order and injects controlled drift of six types.

2. To detect and characterise concept drift using ADWIN, Page–Hinkley, PSI and MMD, individually and in fusion, and to summarise each drift by its magnitude, persistence, unfamiliarity and recurrence.

3. To design an Evolution Controller that selects among no-op, incremental learning, memory replay, strategic sample selection and full retraining.

4. To evaluate every candidate model on independent recent and historical data and to promote it only through a safety gate with versioned registry and automatic rollback.

5. To compare EVO-X experimentally against static and fixed-strategy baselines using predeclared statistics and to report the outcome honestly, including where it does not help.

5 Proposed Method/Algorithm

This chapter restates the problem formally, describes the idea of EVO-X and its architecture with a worked example, and presents the analysis and design of the system.

5.1 Problem Definition

Inputs. (i) A stream of time-ordered batches $B_1, B_2, \ldots$ of network flows, each flow described by a numeric feature vector x and carrying a class label y (benign or an attack family); labels of batch t become available only after a delay of k batches. (ii) An initial deployed model M_1 trained on a reference window. (iii) Role-separated data for evaluation: proxy, gate, monitor and reporting holdouts. (iv) A set of adaptation strategies $\mathcal{A} = \{\text{No-Op}, \text{Incremental}, \text{Replay}, \text{Strategic}, \text{Retrain}\}$ and resource budgets for training time and latency.

Outputs. A sequence of deployed model versions M_v, together with a log recording for every drift event the drift signature, the chosen strategy, the candidate's evaluation, the gate decision and any rollback.

Objective. Keep the predictive quality of the deployed model high on both recent and historical traffic over the whole stream while performing few updates, spending little adaptation cost and never promoting a harmful candidate. For a policy π that produces the model M_t^π deployed at time t this can be written as

$$\max_{\pi} \; \frac{1}{T} \sum_{t=1}^{T} \tfrac{1}{2}\big(F1_t^{\text{recent}}(M_t^\pi) + F1_t^{\text{hist}}(M_t^\pi)\big) \; - \; \lambda \, \text{Cost}(\pi), \tag{1}$$

subject to the promotion constraints of the safety gate given in Section 5.4.4.

Constraints. Labels arrive late and may be partial; a candidate must be judged only on data it was not trained on; training time and inference latency must stay within budget; the active model must remain unchanged if any check fails.

Assumptions. Labels eventually become available for adaptation; the monitored signals (error rate, feature distributions) reflect real changes in traffic; the holdout data are representative of recent and historical traffic; experiments replay public benchmarks, not live networks.

Key challenges. Detecting drift without immediate labels; distinguishing temporary blips from lasting change; balancing plasticity against forgetting; estimating the value of an update cheaply before

paying for it; avoiding leakage between training and evaluation data; and choosing thresholds and weights without tuning them on the test data.

5.2 Proposed Idea/System

EVO-X replaces "drift detected → retrain" with a decision-and-evaluation pipeline of seven cooperating stages. *Detection* produces an alarm; *analysis* turns it into a drift signature; the *controller* selects a strategy; the *adaptation engine* builds a candidate; *evaluation* scores the candidate on independent holdouts; the *safety gate* promotes or rejects it; and *lifecycle management* versions, monitors and, if needed, rolls back the model. The underlying classifier is a replaceable component. The ideas that are new compared with the reviewed work are: the explicit, logged two-stage choice among several adaptation strategies; the rule that a candidate is never deployed before an independent check; the fail-closed behaviour of the gate; and the tamper-evident versioning with monitored rollback, all applied to network-flow IDS.

5.3 System Architecture

Figure 3 shows the end-to-end architecture. Data from the benchmark datasets and the synthetic drift generator are streamed in time order through a preprocessing stage to the currently deployed IDS. The drift engine monitors the stream and hands alarms to the drift analyzer; the Evolution Controller picks a strategy; the adaptation engine produces a candidate; the evaluator scores it; the safety gate decides; and the registry and evolution history record everything. A dashed feedback path lets stored drift signatures support recurrence matching.

Drift detection. ADWIN monitors the per-flow error bits and Page–Hinkley monitors the per-batch error rate; both see labels only after the label delay. The PSI detector compares a sliding window of the last few batches with the reference using ten reference-quantile bins per feature and aggregates the per-feature values as the mean of the five largest, so that a shift in a few features is not diluted by many unchanged ones. The MMD detector uses a Gaussian (RBF) kernel with the median-distance bandwidth on subsamples; its alarm threshold is calibrated from windows drawn from a disjoint half of the reference data. An extra label-free signal applies ADWIN to the prediction confidence. Each detector reports a score

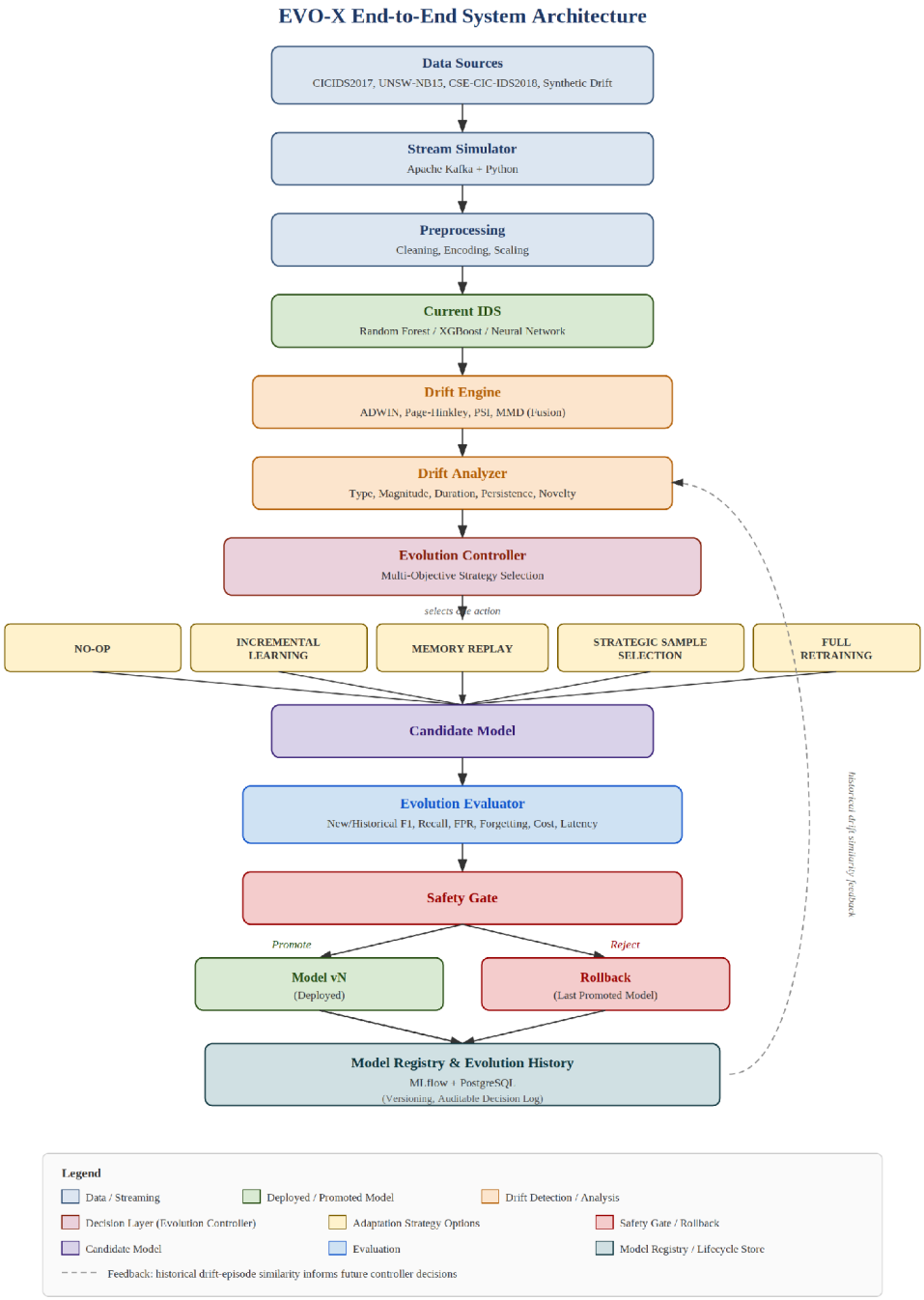


Figure 3: End-to-end architecture of EVO-X.

normalised so that 1.0 is its own alarm threshold, and the scores are fused:

$$D_{\text{fused}} = \sum_i w_i \, \min(s_i, 1), \qquad \sum_i w_i = 1, \tag{2}$$

with an alarm on the rising edge of $D_{\text{fused}} \geq \theta$ and a re-arming rule so that one drift gives one alarm. PSI for one feature is $\sum_b (p_b^{\text{window}} - p_b^{\text{ref}}) \ln\!\left(p_b^{\text{window}} / p_b^{\text{ref}}\right)$ over bins b, and the unbiased squared MMD between samples X and Y is $\text{MMD}^2 = \frac{1}{n(n-1)} \sum_{i \neq j} k(x_i, x_j) + \frac{1}{m(m-1)} \sum_{i \neq j} k(y_i, y_j) - \frac{2}{nm} \sum_{i,j} k(x_i, y_j)$.

Drift analysis. For each alarm the analyzer waits a few batches, never looking beyond its observation point, and builds a *drift signature*: magnitude (the mean of the PSI and MMD scores in units of their own thresholds), persistence (the longest run of batches above threshold), duration, an "ended" flag showing that the shift has already disappeared, a type guess (abrupt, gradual, temporary or recurring), a confidence-based unfamiliarity score, the per-feature PSI vector, and a recurrence score given by the cosine similarity of that vector with stored past signatures. The unfamiliarity score is only a hint: development tests showed that it also rises under plain covariate shift, so it never selects an action on its own.

Adaptation strategies. *No-Op* returns the parent model unchanged. *Incremental* updates the model with recent data: for Random Forest [14] by pruning part of the ensemble and growing new trees, for XGBoost [15] by continuing boosting, which is available only if the set of classes has not changed. *Replay* trains on recent data mixed with a bounded, class-balanced memory of earlier data. *Strategic Selection* is inspired by SSF and keeps the samples with the largest error and lowest confidence, keeps every class represented and expires stale memory. *Full Retrain* trains from scratch on recent data only. Every strategy produces a candidate that records its parent version, strategy, a hash of its training snapshot and its measured resource use; none of them changes the active model.

Evolution controller. Selection is done in two stages. Stage one applies eligibility rules (Algorithm 2) to the signature, shrinking the five strategies to an admissible set; No-Op is always admissible. Stage two ranks the admissible set by a weighted utility,

$$U(a) = w_1 \, \widehat{\Delta F1}(a) - w_2 \, \widehat{\text{Forg}}(a) - w_3 \, \widehat{\text{Cost}}(a) - w_4 \, \widehat{\Delta\text{FPR}}(a) - w_5 \, \widehat{\text{Lat}}(a), \tag{3}$$

$$a^* = \arg\max_{a \in E} U(a), \tag{4}$$

where the estimates of gain, forgetting and false-positive change come from a cheap proxy evaluation on a small holdout, and cost and latency come from measured per-strategy profiles. Ties are broken towards

No-Op and then towards the cheaper action. Because a^* is only a provisional choice, the candidate it yields is still subject to the full evaluation and the safety gate.

Worked example (illustrative numbers, not measured). Suppose an alarm gives a signature with magnitude 4.2 (above $\tau_{\text{high}} = 3$), persistence 12 batches (above $\tau_{p,\text{long}} = 10$), recurrence 0.3 and no "ended" flag. The rules admit $E = \{\text{Retrain}, \text{Replay}, \text{No-Op}\}$. Take weights $w = (0.4, 0.2, 0.2, 0.1, 0.1)$ and normalised proxy estimates as in Table 4. The utilities are $U(\text{Retrain}) = 0.4(0.12) - 0.2(0.10) - 0.2(0.10) - 0.1(0.01) - 0.1(0.10) = -0.003$, $U(\text{Replay}) = 0.4(0.10) - 0.2(0.02) - 0.2(0.05) - 0.1(0.005) - 0.1(0.10) = +0.0155$ and $U(\text{No-Op}) = 0$, so Replay is selected and its candidate goes to evaluation. The example also shows a property that matters later: if the estimated gains were smaller, the cost terms would dominate and No-Op would win, so the scale of the terms and the weights decide how willing the controller is to update.

Table 4: Illustrative proxy estimates for the worked example (invented for explanation only).

Action	$\widehat{\Delta F1}$	$\widehat{\text{Forg}}$	$\widehat{\text{Cost}}$	$\widehat{\Delta\text{FPR}}$	$\widehat{\text{Lat}}$
Retrain	0.12	0.10	0.10	0.010	0.10
Replay	0.10	0.02	0.05	0.005	0.10
No-Op	0	0	0	0	0

Independent evaluation and data roles. To avoid leakage, every original source flow is assigned to exactly one role, even if it appears several times in the stream under different drift offsets: 50% training, 15% proxy estimation, 15% gate evaluation, 10% post-deployment monitoring and 10% final reporting (Figure 4). Candidates never train on proxy, gate, monitor or reporting rows, and reporting rows influence nothing at run time; they are used only for the final metrics. Historical validation samples are split by class across the four evaluation roles.

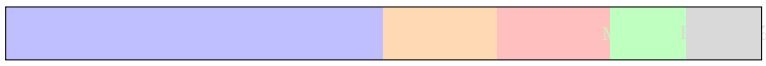


Figure 4: Role assignment of source flows. Each original flow belongs to exactly one role.

Safety gate. The gate promotes a candidate only if all six checks pass:

$$\Delta F1_{\text{new}} \geq \tau_1, \quad \Delta F1_{\text{hist}} \geq -\tau_2, \quad \text{Forg} \leq \tau_3, \quad \Delta\text{FPR} \leq \tau_4, \quad \text{Train} \leq \tau_5, \quad \text{Lat} \leq \tau_6. \tag{5}$$

A missing, non-finite or physically invalid metric causes rejection (fail-closed), and rejection leaves the active model unchanged. The thresholds are not guessed: they are selected from a predeclared grid of 48 configurations using validation scenarios and independent audit outcomes, and are frozen before test scenarios are built.

Registry, monitoring and rollback. Each policy owns a registry of versioned, checksum-verified model artifacts and an append-only, hash-chained journal of events (candidates, decisions, parameters, drift context and pointer changes). Replaying the journal reconstructs which version is deployed; a damaged journal makes the system fail closed. Versions move through the states CANDIDATE, DEPLOYED, SUPERSEDED, REJECTED and ROLLED_BACK. After promotion the active model is replaced in one atomic step, and a monitor compares it with its frozen parent on fresh labelled rows and an independent historical anchor; if performance or false-positive rate degrades, the parent is restored. After the experiment, candidates and parents are also shadow-scored on ten future batches so that a promoted or rejected update can be classified as harmful (it would lower macro-F1 by more than 0.03) or unnecessary (its gain is below 0.01).

5.4 System Analysis and Design

5.4.1 Requirement Specification

The functional and non-functional requirements are listed in Tables 5 and 6.

Table 5: Functional requirements.

ID	Requirement
FR1	Replay benchmark flows as time-ordered batches through a producer/consumer pipeline whose predictions equal direct in-memory scoring.
FR2	Generate six drift scenario types and a drift-free control with recorded ground truth, from disjoint validation and test pools.
FR3	Detect drift with ADWIN, Page–Hinkley, PSI and MMD and fuse their scores.
FR4	Produce a drift signature for each alarm without looking beyond the observation point.
FR5	Select one of five strategies by eligibility rules and utility ranking and log every decision with its reasons.
FR6	Generate candidate models that never replace the active model by themselves.
FR7	Evaluate candidates on independent recent and historical holdouts.
FR8	Apply a fail-closed safety gate with configurable thresholds.
FR9	Version every model, keep a tamper-evident event journal, monitor promoted models and roll back on degradation.
FR10	Compare ten policies under identical data, labels and alarms and report predeclared statistics.

5.4.2 Flowcharts / DFDs / UML / Functional Block Diagram

Figure 5 gives the use-case diagram: the operator or researcher views the dashboard and configures gate thresholds, while the automated Evolution Controller monitors traffic, detects and analyses drift, selects

Table 6: Non-functional requirements.

Property	Requirement
Reproducibility	Seeds, configurations, data hashes and thresholds are stored with each run; results can be regenerated.
Leakage control	Source-group data roles; test-family scenarios refused during tuning.
Auditability	Every promotion, rejection and rollback is recorded and the journal is hash-chained.
Safety	Any missing or invalid metric rejects a candidate; the active model is never modified by a rejection.
Portability	The classifier is replaceable (Random Forest, XGBoost) behind one interface; datasets enter through adapters.
Resource limits	Training under 30 s and inference under 100 ms per 1000 flows in the frozen configuration.
Honesty of reporting	Synthetic or quick runs are labelled and cannot feed final tables.

a strategy, generates and evaluates a candidate, promotes or rejects it and may roll back a version. Figure 6 shows the main classes: the drift engine and analyzer produce a drift signature; the evolution controller selects an adaptation strategy (an abstract class with five concrete subclasses); a candidate model records its parent version; the evaluator scores candidates; the safety gate decides; and the model registry and evolution history store the outcome. Figure 7 is the activity diagram of one evolution cycle and Figure 8 is the corresponding sequence diagram.

5.4.3 Design and Test Cases

Correctness of the mechanisms is verified by automated tests (207 tests in the Phase A–C implementation at the time of writing). The test cases most relevant to the design are listed in Table 7. These tests show that the mechanisms behave as designed; they do not measure detection performance, which is the subject of Chapter 6.

5.4.4 Algorithms and Pseudo Code

Algorithm 1 is the main evolution loop, Algorithm 2 the eligibility rules and Algorithm 3 the safety gate.

The implemented defaults are $\tau_{\text{low}} = 1$, $\tau_{\text{high}} = 3$, $\tau_{p,\text{short}} = 2$, $\tau_{p,\text{long}} = 10$, $\tau_{\text{rec}} = 0.9$ and $\tau_{\text{nov}} = 0.2$. **[TODO: confirm the values frozen for the final run]**

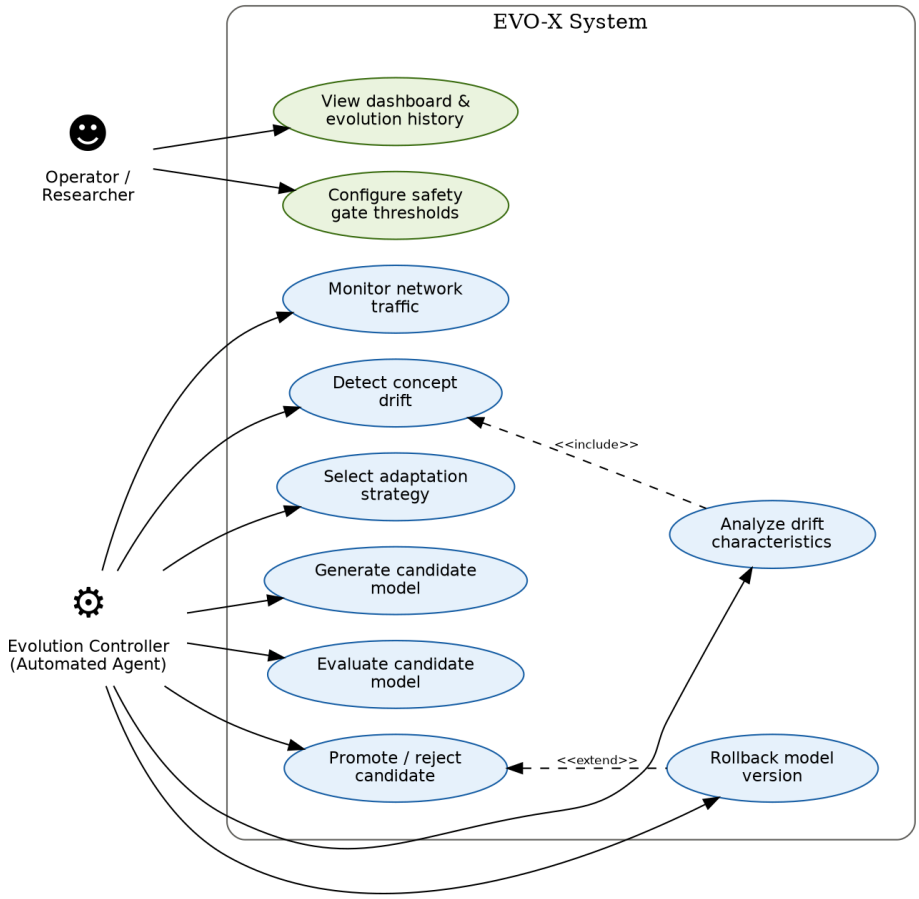


Figure 5: Use-case diagram of EVO-X.

5.4.5 UI/UX Wireframes where applicable

The system is a research framework driven from the command line, and its results are stored as files. A dashboard is planned for the final phase of the project so that a non-specialist reviewer can watch drift detection, controller decisions and rollbacks. Figure 9 is the wireframe of the planned replay-driven dashboard; it has not been implemented yet. **[TODO: replace or remove this wireframe depending on whether the dashboard is built]**

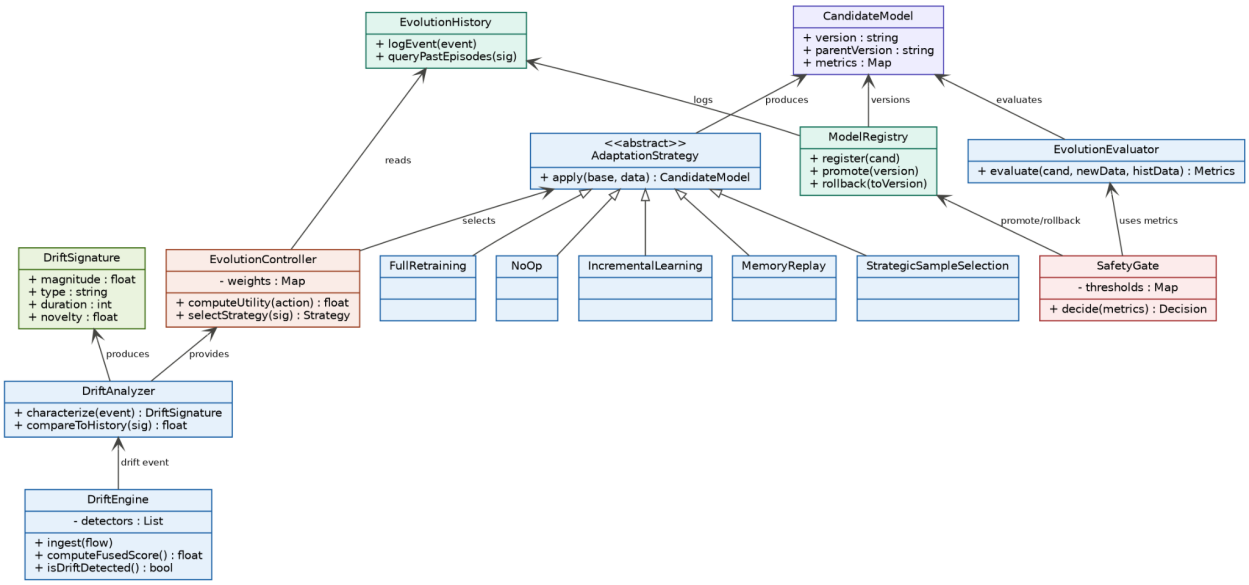


Figure 6: Class diagram of the main components.

6 Performance Study

This chapter describes the experimental environment and metrics, reports the results obtained so far on real CICIDS2017 flows, analyses them, and states the limitations. The results reported here come from a first, deliberately small *quick run* whose purpose was to confirm that the complete comparison workflow runs end to end on real data. They are preliminary and must not be read as a final ranking of the policies. The full multi-seed experiment is described in Section 6.2.

6.1 Implementation/Simulation Environment

Software and justification. The system is written in Python because it has the dominant ecosystem for machine learning and streaming glue code. Scikit-learn provides the Random Forest [14], chosen as a fast, interpretable baseline; XGBoost [15] provides a stronger, industry-realistic tabular model so that portability across algorithms can be examined. River [16] provides ADWIN; Page–Hinkley, PSI and MMD are implemented in the project so that their internal statistics can be observed and tested. Apache Kafka is supported as the streaming backbone because it gives a realistic decoupled producer/consumer arrangement, and an in-memory transport with the same wire format is used as a fallback so that experiments never depend on a running broker. Heavier infrastructure (Spark, Kubernetes) was deliberately avoided as unnecessary for a three-to-four-person academic project. Tests are run with PyTest. **[TODO: add hardware used**

Table 7: Representative test cases.

Area	Test	Expected
Streaming	Batch encoded, sent and decoded; delivery repeated; batch missing	Bit-identical data; duplicate skipped; gap raises error
Streaming	Scorer reads batch without labels	Labels never decoded
Streaming	Predictions through pipeline vs. direct loop	Identical predictions and confidences
Drift lab	Truth vs. generated stream; recurring segments; novel class first appearance	Truth matches data; recurrence repeats; first batch correct
Drift lab	Validation and test pools; magnitude levels	Pools disjoint; same drift at three strengths
Detectors	Step and ramp signals with known change point; stationary noise	Alarm shortly after change; few false alarms
Analyzer	Future data corrupted beyond observation point	Signature unchanged (no look-ahead)
Gate	Values exactly on and just beyond each threshold; missing metric	Boundary behaviour correct; reject on missing
Registry	Reject then promote then restore; reopen registry; damaged journal	Parent predictions restored exactly; state recovered; fail closed
Data roles	Source flow reused under different offsets	Stays in one role

(CPU, RAM, GPU if any), Python and library versions, and run time of the final experiment]

Data. The source flows are real CICIDS2017 records [8]. The CSV files did not provide usable timestamps, so the stream follows capture-file order. Drift is created by controlled re-mixing of class pools and by bounded additive perturbations of informative features; it is therefore *injected* drift over real flows, not naturally observed attack transitions.

Parameters varied. Table 8 lists what was varied. The quick run used one validation seed, one test seed, the medium magnitude and the Random Forest classifier. The full protocol uses three validation seeds, at least six test seeds, three magnitudes and both classifiers.

Metrics. Accuracy counts every correctly classified row and is dominated by common classes, so it is shown but not used as a headline metric. For class c, $F1_c = 2P_cR_c/(P_c + R_c)$ with precision P_c and recall R_c; *macro-F1* is the unweighted mean of $F1_c$ over classes. The false-positive rate is $\mathrm{FPR} = FP/(FP+TN)$ for benign traffic flagged as attack. *Historical forgetting* is the drop in per-class F1 on a historical holdout relative to the best earlier value, averaged over the classes concerned. *Updates per scenario* is the number of promoted models per policy-scenario run. *Training plus proxy-search time* is the wall-clock time spent building and estimating candidates. For detectors, *detection delay* is the number of batches between the true change and the alarm, and the *false-alarm rate* is the number of alarms outside any truth window per

Algorithm 1 EVO-X evolution loop

deploy M_1; initialise registry, journal and signature store
for each batch B_t **do**
    predict B_t with the deployed model; release labels of B_{t-k}
    update detectors; compute D_{fused}
    if alarm raised **then**
        wait for the observation window; $\sigma \leftarrow$ CHARACTERISE(history up to now)
        $E \leftarrow$ ELIGIBLE(σ); $a^* \leftarrow \arg\max_{a \in E} U(a)$
        if $a^* \neq$ No-Op **then**
            $M' \leftarrow$ ADAPT($M_v, a^*,$ released labels)
            metrics $\leftarrow$ EVALUATE($M', M_v,$ recent and historical holdouts)
            if GATE(metrics) passes **then**
                promote M' as new deployed version; start monitor
            else
                reject M'; keep M_v
            end if
        end if
        append decision record to the journal
    end if
    if monitor reports degradation **then**
        restore the parent version
    end if
end for

Algorithm 2 Eligibility rules (thresholds are parameters; implemented defaults in text)

Require: magnitude m, persistence p, ended flag, recurrence r, unfamiliarity n, budget
if $m < \tau_{\text{low}}$ **or** $p < \tau_{p,\text{short}}$ **or** ended **then**
    $E \leftarrow$ {No-Op}
else if $r \geq \tau_{\text{rec}}$ **then**
    $E \leftarrow$ {Replay, Incremental, No-Op}
else if $n \geq \tau_{\text{nov}}$ **and** $m \geq \tau_{\text{high}}$ **then**
    $E \leftarrow$ {Retrain, Strategic, No-Op}
else if $m \geq \tau_{\text{high}}$ **and** $p \geq \tau_{p,\text{long}}$ **then**
    $E \leftarrow$ {Retrain, Replay, No-Op}
else
    $E \leftarrow$ {Incremental, Strategic, Replay, No-Op}
end if
remove unavailable actions and actions whose estimated cost exceeds the budget

1000 batches. A promoted update is *harmful* if shadow-scoring on ten future reporting batches shows that it would lower macro-F1 by more than 0.03 relative to the frozen parent, and *unnecessary* if its gain is below 0.01; empty or truncated horizons are flagged as censored. For statistics, results are averaged within each seed and compared with a paired Wilcoxon signed-rank test [18] with Holm correction [17]; the energy-saving claim H2 additionally uses a non-inferiority margin of 0.02 macro-F1. With fewer than six paired seeds no test is run.

Hypotheses. H1: strategy selection improves detection relative to fixed policies. H2: EVO-X performs fewer updates than always-retrain while macro-F1 stays within 0.02 of the best fixed policy. H3: the safety gate reduces harmful promotions relative to the no-gate ablation. H4: EVO-X shows less historical

Algorithm 3 Safety gate (fail-closed)

Require: candidate metrics μ, thresholds $\tau_1, \ldots, \tau_6$
if any required metric is missing, non-finite or invalid **then**
    return REJECT
end if
if $\Delta F1_{\text{new}} \geq \tau_1$ **and** $\Delta F1_{\text{hist}} \geq -\tau_2$ **and** Forg$\leq \tau_3$ **and** ΔFPR$\leq \tau_4$ **and** Train$\leq \tau_5$ **and** Lat$\leq \tau_6$ **then**
    return PROMOTE
else
    return REJECT
end if

Table 8: Parameters varied in the experiments.

Parameter	Values
Scenario type	abrupt, gradual, recurring, temporary, incremental, novel-class, and a stationary control
Magnitude	low, medium, high (quick run: medium only)
Policy	Static; Always Retrain; Always Incremental; Always Replay; Active-incremental; SSF-inspired; EVO-X; EVO-X without gate; EVO-X rules-only; EVO-X utility-only
Classifier	Random Forest (quick run); XGBoost (planned)
Label delay	k batches, same for all policies (default proposal $k = 2$)
Controller weights w_i	$\{0, 0.25, 0.5, 1\}$, normalised, selected on validation scenarios
Gate thresholds	48-point grid; frozen values in Table 9
Seeds	validation 3; test ≥ 6 (quick run: 1 each)

forgetting than recent-only full retraining. Each is reported as supported, not supported or inconclusive under predeclared rules; synthetic and quick runs are always inconclusive.

6.2 Results and Analysis

Experiment of the quick run. A controller configuration chosen on validation data in an earlier phase was reused. Gate thresholds were then selected on validation scenarios (validation seed 1001) from the 48-point grid and frozen (Table 9). Ten policies were then replayed on seven held-out scenarios (the six drift types and the stationary control, medium magnitude, test seed 2001), giving $7 \times 10 = 70$ policy-scenario runs. All policies shared the same initial model, source flows, label access and alarms.

Table 9: Safety-gate thresholds selected on validation data and frozen for the quick run.

Check	Frozen threshold
Minimum gain on new data	0.02
Maximum drop on historical data	0.02
Maximum forgetting	0.03
Maximum FPR increase	0.005
Training time budget	30 s
Latency budget	100 ms per 1000 flows

Comparison of policies. Table 10 and Figure 10 show the averages over the seven scenarios. Error bars in the figure are the spread across scenarios, not confidence intervals, and the run has a single test seed, so the comparison cannot establish statistical superiority.

Table 10: Quick-run results on real CICIDS2017 flows with injected drift (mean over seven scenarios, one test seed, Random Forest). Preliminary.

Policy	Accuracy	Macro-F1	Forgetting	Updates/scenario
Active incremental	92.44%	0.637	0.132	2
Always incremental	91.12%	0.626	0.067	2
Always replay	89.90%	0.618	0.026	2
EVO-X	88.82%	0.599	0.000	0
Static (no update)	88.82%	0.599	0.000	0
Always retrain	88.50%	0.569	0.182	2
SSF-inspired	81.67%	0.468	0.331	2

The three EVO-X ablations (no gate, rules-only, utility-only) also made zero promotions and plotted at the static macro-F1 in Figure 10.

Observations. *(1) Updating is not automatically beneficial.* Relative to the static model, active-incremental raised macro-F1 by 0.038 (from 0.599 to 0.637, about 6%) and accuracy by 3.6 percentage points, always-incremental raised macro-F1 by 0.027 and replay by 0.019. In contrast, always-retrain lowered macro-F1 by 0.030 and the SSF-inspired policy by 0.131 (about 22%), and these two also forgot the most (0.182 and 0.331). A plausible reason is that retraining on recent data only discards knowledge of earlier classes, and that the strategic selection and expiry steps of the SSF-inspired policy can discard classes that remain relevant; this explanation fits the forgetting figures but was not tested separately in this run. *(2) Memory limits forgetting.* Among the fixed update policies, replay had the lowest forgetting (0.026), consistent with its use of remembered samples. *(3) EVO-X matched static and did not improve on it.* EVO-X made no promotions, so its macro-F1, accuracy and forgetting equal those of the static model. Zero forgetting and zero rollbacks here follow directly from not changing the model; they do not show that the safety machinery handled a risky update, because no update was promoted. *(4) The "70 promotions" in the summary file is a total over all policies.* Five fixed policies each deployed 14 candidates ($5 \times 14 = 70$); EVO-X variants contributed none.

Safety-gate behaviour. The gate rejected six candidates in this run, all under the rules-only EVO-X variant. The no-gate and utility-only variants also made zero promotions, which indicates that for those variants the controller selected No-Op, or that no valid candidate existed, before the gate was reached. The candidate-event log and decision records should be inspected to confirm which; this is the main open question raised by the quick run, because a controller that never updates is trivially safe but also cannot demonstrate adaptation. Possible causes are drift-size thresholds that are too high, utility weights or cost

scaling that make the cost terms dominate (as in the worked example of Section 5.3), or unreliable proxy estimates. Until this is examined, no statement can be made about whether the gate improves safety or about whether strategy selection helps.

Cost. The fixed update policies spent roughly 0.3 to 0.6 seconds per scenario on training, while the EVO-X variants spent roughly 2.4 to 2.8 seconds on training plus proxy search (values read from Figure 10). EVO-X therefore pays for evaluating possible candidates even when it chooses not to deploy one. All times stay far below the 30 s training budget of the gate.

Hypotheses. With one test seed and seven scenarios, all four hypotheses are labelled inconclusive. This is the correct label: the run confirms the workflow, not a conclusion about which policy is better in general.

Observations during development. While building and testing the drift detectors on a synthetic stream (used only to check the software, not to report performance) several qualitative lessons were learned that shaped the design. ADWIN did not fire when fed batch-average error rates but works when fed individual error bits, so it is fed per flow. MMD with a null-quantile threshold has a small, expected false-alarm rate by construction. Confidence-based unfamiliarity rises under covariate shift as well as for new classes, so it is used only as a hint in the rules. PSI and MMD alone can miss a small new-class share that error-based detectors catch once labels arrive, which is the reason for using several detectors. These observations come from synthetic data and are not claimed as results on real traffic.

Full experiment plan and remaining results

[TODO: insert final tables from the full real-data run: (a) detector comparison on real flows (detection rate, delay, false alarms per 1000 batches), (b) per-scenario policy comparison with mean and standard deviation over at least six test seeds and three magnitudes, (c) paired Holm-corrected statistics and hypothesis status, (d) counterfactual harmful/unnecessary counts and gate-sensitivity sweep, (e) Random Forest versus XGBoost and UNSW-NB15 generalisation]

6.3 Summary of performance study

On the evidence so far, the comparison workflow, safety gate and lifecycle machinery run correctly end to end on real CICIDS2017 flows, and the first results show that update policies differ greatly: some improved the static model modestly, while others reduced macro-F1 and caused substantial forgetting. EVO-X was safe only in the trivial sense that it made no update, and it did not yet demonstrate a benefit. The performance is therefore not shown to be better in all cases or in any specific condition; this must be decided by the full multi-seed experiment. As for practicality, the extra proxy-search time of a few seconds per scenario and the per-candidate evaluation are small compared with the gate budgets, so computational cost does not appear to be a barrier at this scale.

6.4 Limitations

- **Scale of the evidence.** The only real-data comparison so far is a quick run with one test seed, seven scenarios, one magnitude and one classifier; error bars are spread across scenarios, not confidence intervals.
- **Injected drift.** Drift is created by re-mixing and perturbing real flow pools. It is not naturally observed drift, and real drift may be more complex.
- **Stream order.** The CICIDS2017 CSVs lacked usable timestamps, so the stream follows capture-file order and is not a true chronological replay.
- **Dataset age and artefacts.** CICIDS2017, UNSW-NB15 and CSE-CIC-IDS2018 contain known labelling and generation artefacts, so results are relative comparisons and not claims of real-world accuracy.
- **Few classifiers.** Only tabular flow-based models (Random Forest and XGBoost) are used; results may not carry over to graph-based or sequence-based IDS.
- **Threshold sensitivity.** Controller weights and gate thresholds depend on the domain and risk tolerance; the study reports sensitivity but cannot claim universally optimal values.
- **Baselines.** The SSF-inspired and active-learning-inspired policies are approximations written for this project, not reproductions of the original systems.
- **Harm measurement.** Harm is measured by a finite-horizon comparison against the frozen parent, not by replaying the entire alternative trajectory.

- **No live deployment.** All experiments are replays; no real production network was used.

7 Conclusions and Future Work

The objective of this study was to design, implement and evaluate a framework that governs how a flow-based intrusion detector evolves under concept drift, by detecting and characterising drift, choosing among adaptation strategies, testing every candidate model on independent data and deploying it only through a fail-closed safety gate with monitored rollback. The framework was built and verified component by component, and a first quick comparison of ten policies on real CICIDS2017 flows with injected drift confirmed that the whole workflow runs end to end. The findings were partly expected and partly unexpected: it was expected that fixed-update policies would behave differently, but it was notable how far the outcomes diverged, with incremental and replay updates modestly improving on the static model while full retraining and the SSF-inspired policy lowered macro-F1 and forgot much more, and it was unexpected and unsatisfactory that EVO-X itself made no promotions and so matched the static model without demonstrating any gain. The objective of building and verifying the governed-evolution framework is therefore achieved, but the objective of showing that it outperforms fixed policies is not yet achieved, and all four hypotheses remain inconclusive until the planned multi-seed, multi-magnitude experiments, preceded by an examination of why the controller keeps choosing no update, have been completed.

Future Work

- Examine the controller's decision records to find why it chose no update, and re-tune its thresholds and weights on validation scenarios if it is too conservative.
- Run the full protocol (at least six test seeds, three magnitudes, both Random Forest and XGBoost) and report statistics, harmful and unnecessary updates, and gate sensitivity.
- Evaluate generalisation to UNSW-NB15 and, if time permits, CSE-CIC-IDS2018, after verifying the dataset adapters.
- Build the replay-driven dashboard and a scripted end-to-end demonstration.
- Explore learned (meta-learning or reinforcement-learning) strategy selection using the stored evolution history, gradient-projection forgetting mitigation inspired by RepShield, federated or privacy-preserving adaptation across sites, extension to IoT intrusion data, and robustness against adversarial drift designed to manipulate the controller.

8 Scholarly Contributions

1. **Research papers.** A research paper in IEEE conference format has been drafted from this work; it has not yet been submitted. **[TODO: add title, venue and status (communicated / accepted / published) when known]**

2. **Patents or copyright filed.** None.

3. **Project exhibitions.** **[TODO: list any project exhibition or competition participation, or write "None"]**

4. **Open-source contributions, repository, demo video, deployed product.** Source code, tests and configuration are maintained in the project repository. **[TODO: GitHub repository link, demo video link]**

5. **Certifications relevant to the project.** **[TODO: list relevant certifications of team members, or write "None"]**

9 References

[1] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[2] L. Wang, X. Zhang, H. Su, and J. Zhu, "A comprehensive survey of continual learning: Theory, method and application," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 46, no. 8, pp. 5362–5383, 2024.

[3] **[TODO: authors]**, "Evolving cybersecurity frontiers: A comprehensive survey on concept drift and feature dynamics aware machine and deep learning in intrusion detection systems," *Engineering Applications of Artificial Intelligence*, vol. 137, p. 109143, 2024.

[4] X. Zhang, R. Zhao, Z. Jiang, H. Chen, Y. Ding, E. C. H. Ngai, and S.-H. Yang, "Continual learning with strategic selection and forgetting for network intrusion detection," arXiv preprint arXiv:2412.16264, 2024 (accepted at IEEE INFOCOM 2025).

[5] Y. Liu, W. Niu, H. Li, M. He, X. Ding, J. Gong, R. Yan, Z. Li, and X. Zhang, "RepShield: Robust knowledge representation in continual learning for network intrusion detection," *Computer Networks*, 2026, in press.

[6] J. Ying, M. Zheng, J. Chen, R. Chen, Z. Zhu, and T. Zhu, "METANOIA: A lifelong intrusion detection and investigation system for mitigating concept drift," arXiv preprint arXiv:2501.00438, 2025.

[7] T. M. T. Pham, K. Premkumar, M. Naili, and J. Yang, "Time to retrain? Detecting concept drifts in machine learning systems," in *Proc. 47th IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2025.

[8] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, pp. 108–116, 2018.

[9] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Military Communications and Information Systems Conference (MilCIS)*, pp. 1–6, 2015.

[10] Communications Security Establishment (CSE) and Canadian Institute for Cybersecurity (CIC), "CSE-CIC-IDS2018 on AWS: A realistic cyber defense dataset," 2018. [Online]. Available: https://registry.opendata.aws/cse-cic-ids2018/

[11] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. Data Mining (SDM)*, pp. 443–448, 2007.

[12] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[13] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, "A kernel two-sample test," *Journal of Machine Learning Research*, vol. 13, pp. 723–773, 2012.

[14] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[15] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pp. 785–794, 2016.

[16] J. Montiel, M. Halford, S. M. Mastelini, *et al.*, "River: Machine learning for streaming data in Python," *Journal of Machine Learning Research*, vol. 22, no. 110, pp. 1–8, 2021.

[17] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[18] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

A Appendix A: Plagiarism Report, AI Similarity Report

This appendix contains the plagiarism and AI similarity report of the project. The similarity index (**plagiarism**) of the report **must not exceed 15%**, and the **AI-generated content similarity must be nil**.

[TODO: attach the plagiarism and AI-similarity reports generated by the institute's approved tool]

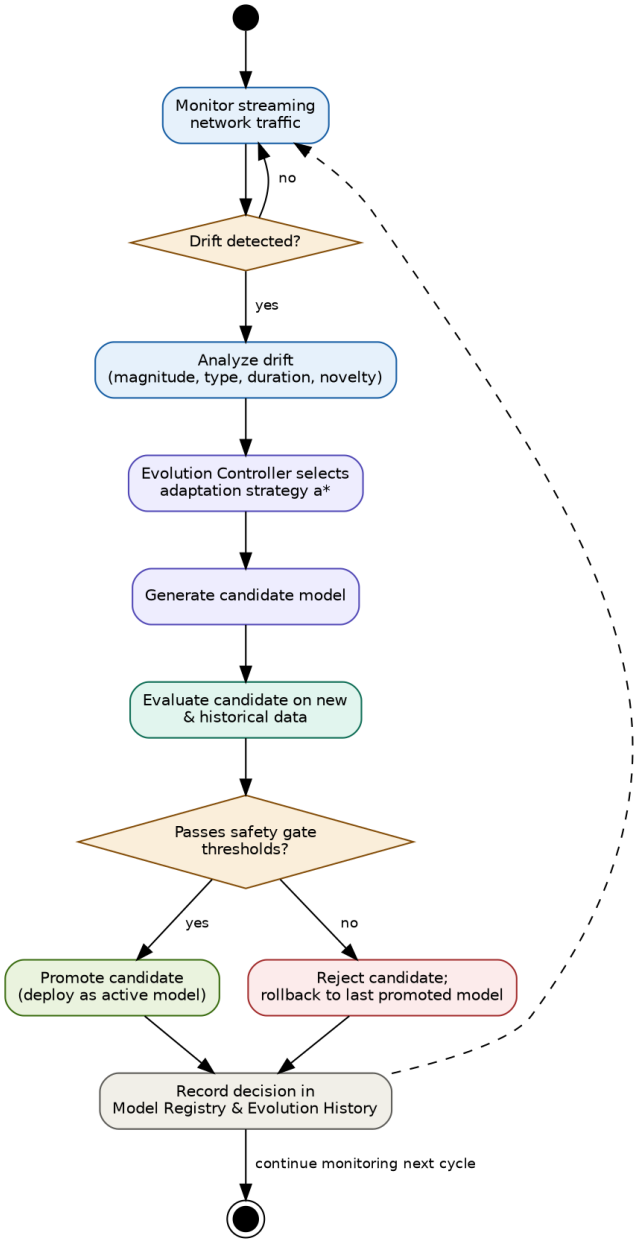


Figure 7: Activity diagram of one evolution cycle.

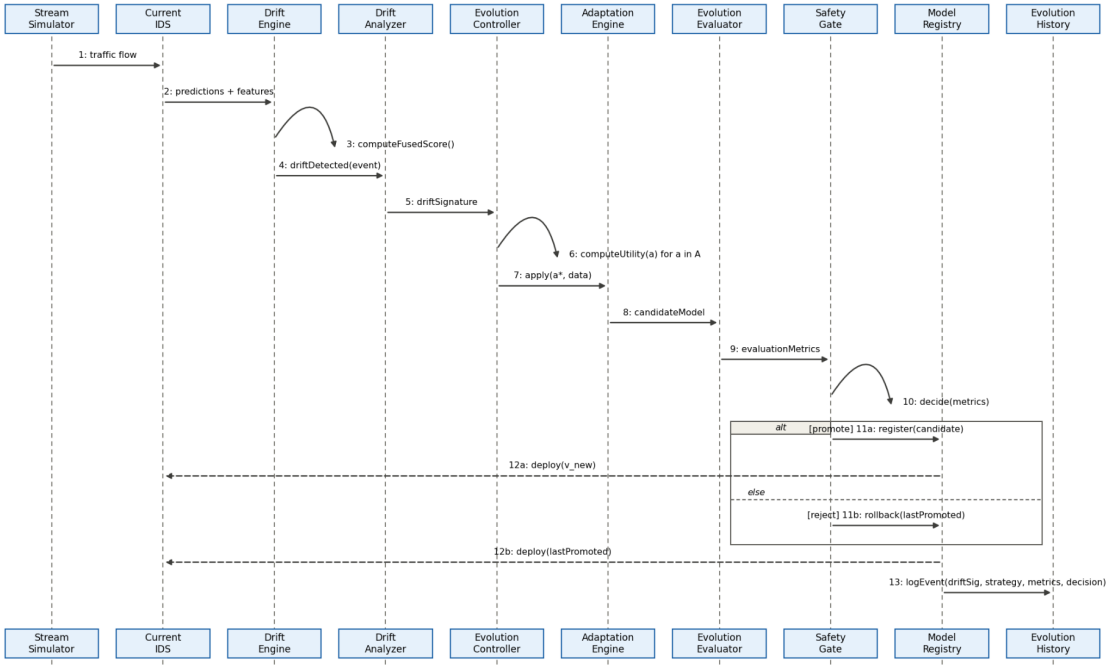


Figure 8: Sequence diagram of drift detection through promotion or rollback.

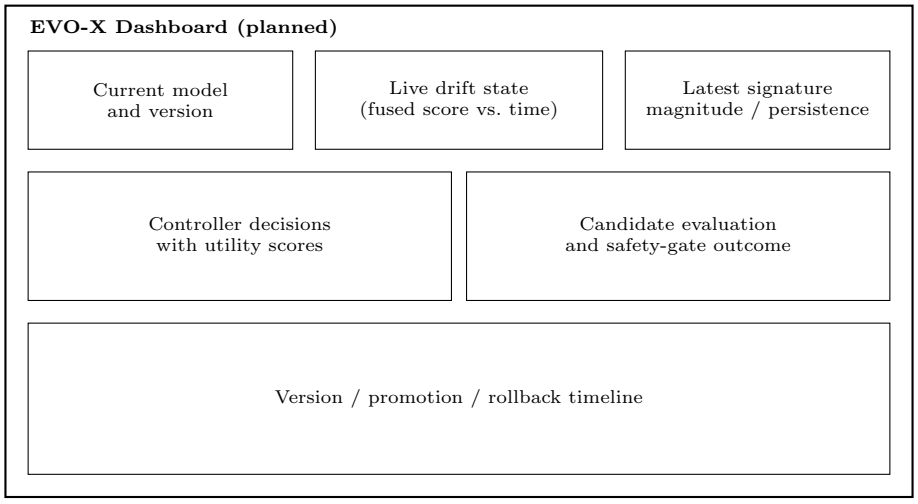


Figure 9: Wireframe of the planned dashboard.

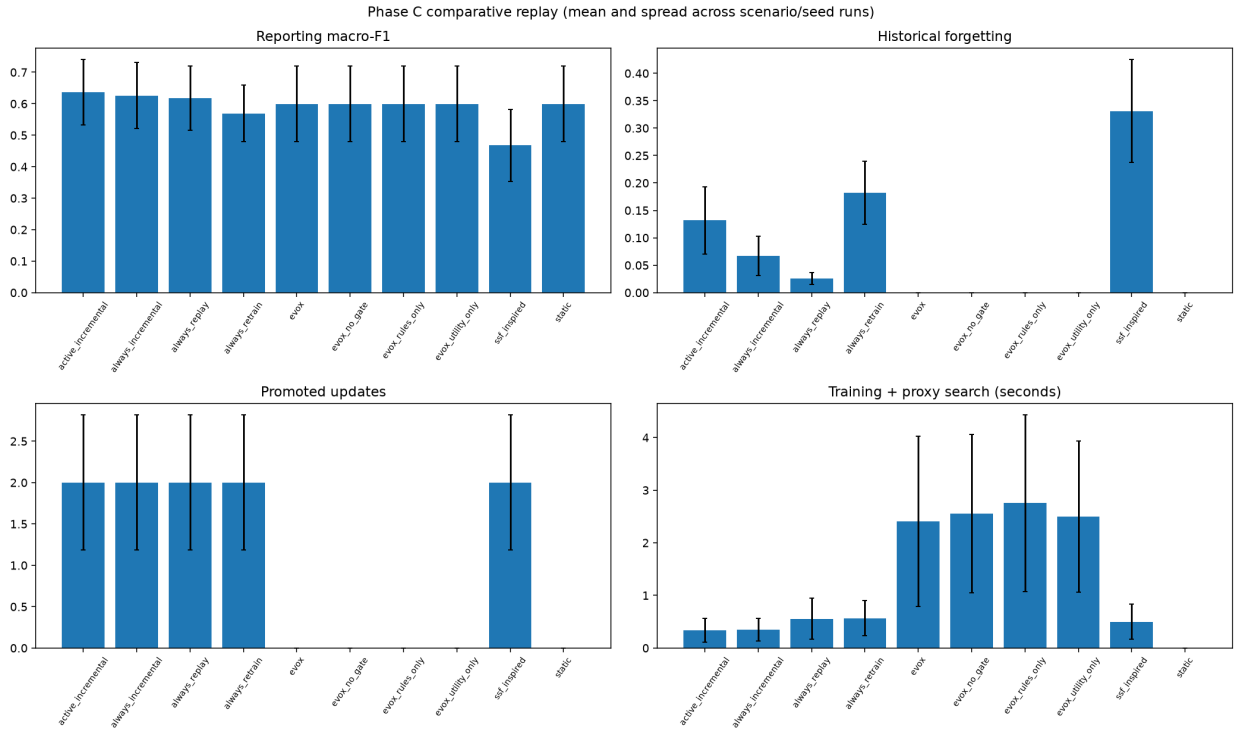


Figure 10: Quick-run comparison across the ten policies: reporting macro-F1 (top left), historical forgetting (top right), promoted updates (bottom left) and training plus proxy-search time in seconds (bottom right). Bars show the mean over seven scenarios; error bars show spread across scenarios.